CECS 447 Final Project:
I2C Bus Communication System

Jackie Huynh and Dr. Min He

April 20, 2026

Course: CECS 447 (Senior Embedded Systems)
Duration: April 20 — May 13, 2026 (24 days, 3-step workflow)

Timeline:
Step 1: April 20–22 (3 days) — Requirements & System Design
Step 2: April 23 — May 4 (12 days) — Module Design & Implementation
Step 3: May 5–13 (9 days) — Feature Implementation, Integration & Final Submission

---

Executive Summary

In this final project, you will design and implement an **I2C-based sensor fusion system** that integrates multiple I2C devices on a shared communication bus. The system reads color and motion data from sensors, displays information on an LCD, and controls a servo motor in real-time. This project teaches essential embedded systems concepts: multi-device bus management, sensor data processing, real-time constraints, and system integration.

1 Learning Objectives

Upon completion of this project, students will be able to:

1. **Master the I2C Protocol:** Understand I2C bus communication, slave addressing, clock timing, and multi-master arbitration
2. **Manage Multiple I2C Devices:** Design and implement systems with multiple I2C slave devices sharing a single bus
3. **Process Sensor Data:** Read, calibrate, and interpret data from multiple analog and digital sensors
4. **Control Hardware with PWM:** Generate precise PWM signals to control servo motor position
5. **Integrate Complex Systems:** Combine independent modules into a coordinated, real-time system
6. **Debug Embedded Systems:** Use systematic debugging techniques for multi-device systems
7. **Document Technical Work:** Communicate design decisions and test results professionally

2 Project Overview

2.1 System Architecture

The I2C bus system consists of seven independent modules that work together:

- **Timing & Delays (WTIMER):** Wide timer (WTIMER1–5, student choice) provides 1 millisecond delays used throughout the system
- **Communication (UART0):** Sends data to PC for debugging and logging
- **I2C Bus (I2C):** I2C module (I2C1–3, student choice) manages communication with three I2C slave devices
- **Color Sensor (TCS34725):** Detects RGB color values from objects
- **Motion Sensor (MPU6050):** Measures acceleration and tilt angles
- **Display (16x2 LCD):** Shows color names and angle values
- **Motor Control (Servo via PWM):** Hardware PWM module (M0PWM1–7 or M1PWM0–7, student choice) positions servo motor based on sensor input

2.2 System Behavior

The complete system operates as follows:

1. **Initialization:** All modules initialize and verify sensor presence
2. **Continuous Sampling:** Every 100 milliseconds, read color and angle data
3. **Data Processing:** Determine dominant color and servo angle from sensor readings
4. **Output:** Every 1 second, display data on LCD and UART terminal
5. **Control:** Servo motor position responds to IMU tilt in real-time

3 Required Components

Component	Specification	Purpose
TIVA TM4C LaunchPad	TM4C123 microcontroller	Main processor
TCS34725 RGB Color Sensor	I2C interface, address 0x29	Color detection
MPU6050 6-DOF IMU	I2C interface, address 0x68	Tilt/acceleration measurement
16x2 LCD Display	I2C backpack (PCF8574T), address 0x27	Text output
Angular Servo Motor	Standard 5V, 50Hz PWM control	Mechanical actuator
Pull-up Resistors	2× 4.7kΩ resistors	I2C bus termination (SDA, SCL)
Serial Terminal	Putty, Tera Term, YAT, etc.	PC-based debugging interface

4 Project Requirements

4.1 System Interfaces & Peripherals

Interface	Configuration	Details
UART0	115200 baud (student choice, not 57600)	Serial communication with PC for debugging
I2C (I2C1–3)	100 kHz (Standard Mode)	Bus communication for all three I2C devices (I2C0 not allowed)
PWM (M0PWM1–7 or M1PWM0–7)	50 Hz	Servo motor control via pulse width modulation (M0PWM0 not allowed)
Wide Timer (WTIMER1–5)	1 ms resolution	Provides delays for timing-critical operations (WTIMER0 not allowed)

4.2 Code Implementation Requirements

The starter code contains two types of placeholders that must be completed:

- **CONSTANT_FILL** — Replace with correct constant values:
  - I2C slave addresses (0x29, 0x68, 0x27)
  - Register addresses for each sensor
  - Clock dividers and timing constants
  - PWM load and duty cycle values
- **CODE_FILL** — Implement required functionality:
  - I2C initialization and read/write sequences
  - Sensor configuration and data reading
  - LCD output formatting and display control
  - Servo PWM duty cycle calculation
  - Main loop control and timing

Completion Standard: All functions must be fully implemented. Partial implementations or TODOs remaining in code will result in point deductions.

5 Module Specifications

5.1 Module 1: Wide Timer (WTIMER)

Purpose: Provide 1 millisecond delay function used by all other modules
Hardware: 32-bit wide timer peripheral (WTIMER1–5, student choice; WTIMER0 not allowed)
Clock: System clock divided appropriately for 1 ms resolution

Functional Requirements

- `WTIMER_Init()`: Initialize timer for 1 ms delays
- `WTIMER_Delay(ms)`: Block for specified milliseconds
- LED Control: Red LED blinks at 2 Hz in timer mode
- Mode Selection: SW1 toggles between UART test mode and WTIMER mode
- LED Cycling: SW2 cycles through onboard LEDs (Red → Green → Blue)

Test Criteria

- LED blink is regular (no jitter) at exactly 2 Hz
- Delays are within ±5% accuracy
- Mode toggle works reliably
- LED cycling completes in order

5.2 Module 2: UART0 Serial Communication

Purpose: Send debug information and sensor data to PC terminal
Hardware: UART0 peripheral with student-chosen baud rate (not 57600; suggest 115200, 9600, or 38400)
Clock: System clock divided appropriately for chosen baud rate (8 data bits, 1 stop bit, no parity)

Functional Requirements

- `UART0_Init()`: Initialize UART at chosen baud rate (not 57600)
- `UART0_SendChar(c)`: Send single character
- `UART0_SendString(s)`: Send null-terminated string
- `UART0_SendInteger(n)`: Send integer as decimal string
- `UART0_SendFloat(f, decimals)`: Send float with specified precision
- **Timing:** Output updates every 1 second (1 Hz)

Test Criteria

- All characters transmit without corruption
- Terminal displays text correctly at chosen baud rate
- Integer and float formatting correct (use sprintf)
- Output timing is regular (±100 ms tolerance)

5.3 Module 3: I2C Bus Communication

Purpose: Manage all I2C communication with external slave devices
Hardware: I2C module (student choice: I2C1, I2C2, or I2C3; I2C0 not allowed) with three slave devices on shared SDA/SCL lines
Clock: Standard Mode at 100 kHz (External 4.7kΩ pull-ups required on SDA and SCL)

Functional Requirements

- `I2C_Init()`: Initialize I2C module at 100 kHz
- `I2C_Write(addr, reg, data)`: Write single byte to register
- `I2C_Read(addr, reg, count, buffer)`: Read multiple bytes from register
- **Error Handling:** Detect timeout/NAK and return status code
- **Pull-ups:** External 4.7kΩ resistors on SDA and SCL required

Slave Addresses

- TCS34725 Color Sensor: 0x29
- MPU6050 IMU: 0x68
- 16x2 LCD Backpack: 0x27

Test Criteria

- I2C clock exactly 100 kHz (measure with oscilloscope)
- ACK bits present for each address byte
- All devices respond to read/write commands
- Timeout handling prevents system hang

5.4 Module 4: TCS34725 RGB Color Sensor

Purpose: Detect color of objects via RGB values
Hardware: TCS34725 Color Sensor (I2C Address: 0x29) via I2C communication (student-chosen I2C module)
Clock: 2-byte register reads with ID register (0x12) verification returning 0x44

Functional Requirements

- `TCS34725_Init()`: Initialize sensor, detect via ID register
- `TCS34725_Enable()`: Turn on color sensing (ENABLE register)
- `TCS34725_ReadColor(r, g, b)`: Read 16-bit RGB values
- **Sensor Detection:** ID register (0x12) should return 0x44
- **Output:** LED lights up to indicate detected color (Red/Green/Blue)

Register Mapping

- ID Register: 0x12 (should read 0x44)
- Enable Register: 0x80 (bit 0 = sensor enable)
- Color Data: 0x94–0x99 (R, G, B values as 16-bit)

Test Criteria

- Sensor detected correctly on first initialization
- RGB values change when different colors shown to sensor
- Values not garbage (not all 0xFF or 0x00)
- Sensor responds within I2C timeout window

5.5 Module 5: MPU6050 6-DOF Inertial Measurement Unit

Purpose: Measure acceleration and rotation angles
Hardware: MPU6050 IMU Sensor (I2C Address: 0x68) via I2C communication (student-chosen I2C module)
Clock: 16-bit signed integers in 2's complement with WHO_AM_I register (0x75) verification returning 0x68

Functional Requirements

- `MPU6050_Init()`: Initialize sensor, detect via WHO_AM_I register
- `MPU6050_Enable()`: Configure and enable measurements
- `MPU6050_ReadAccel(ax, ay, az)`: Read 3-axis acceleration
- `MPU6050_ReadGyro(gx, gy, gz)`: Read 3-axis rotation rate
- **Sensor Detection:** WHO_AM_I register (0x75) should return 0x68

Register Mapping

- WHO_AM_I Register: 0x75 (should read 0x68)
- Power Management: 0x6B (enable sensor)
- Acceleration Data: 0x3B–0x40 (6 bytes, X/Y/Z)
- Gyroscope Data: 0x43–0x48 (6 bytes, X/Y/Z)

Scale Factors

- Acceleration: $\pm 2g$ range $\rightarrow$ 16,384 LSB/g
- Gyroscope: $\pm 250°/s$ range $\rightarrow$ 131 LSB/(°/s)

Test Criteria

- Sensor detected correctly on initialization
- Z-axis acceleration reads ≈16,384 LSB when stationary (1g gravity)
- Values change when board is moved and rotated
- Data refreshes within 100 milliseconds

5.6 Module 6: 16x2 LCD Display with I2C Backpack

Purpose: Display color names and angle values to user
Hardware: 16x2 LCD Display with I2C Backpack (PCF8574T, I2C Address: 0x27) via I2C communication (student-chosen I2C module)
Clock: 16 characters per row, 2 rows, with display timing matching sensor update frequency

Functional Requirements

- `LCD_Init()`: Initialize LCD via I2C backpack
- `LCD_PrintText(text)`: Display text on LCD
- `LCD_SetCursor(row, col)`: Position cursor
- `LCD_Clear()`: Clear display and home cursor
- **Display Output:** Row 1: First name (startup), then "Color: [NAME]"; Row 2: Last name (startup), then "Angle: [VALUE]°"

Initial Behavior (Startup)

1. Display first name on Row 1
2. Wait 1 second
3. Display last name on Row 2
4. Wait 1 second
5. Clear display
6. Repeat

Full System Behavior

- Row 1: "Color: RED" / "Color: GREEN" / "Color: BLUE"
- Row 2: "Angle: 45°" (or current angle value)
- Updates every 1 second

Test Criteria

- Text appears without garbage characters
- Both rows display clearly and independently
- Display timing follows specification exactly
- Backlight is on and readable

5.7 Module 7: Servo Motor Control via PWM

Purpose: Control servo motor position based on IMU angle
Hardware: PWM module (student choice: M0PWM1–7 or M1PWM0–7; M0PWM0 not allowed) with servo motor connected
Clock: 50 Hz frequency (20 millisecond period) with 1.0–2.0 ms pulse width for ±90° range control

Functional Requirements

- `PWM_Init()`: Configure PWM at 50 Hz frequency
- `Servo_SetAngle(angle)`: Set servo to specified angle ($-90°$ to $+90°$)
- **Angle Mapping:**
  - $-90° \rightarrow$ 1.0 ms pulse (5% duty cycle)
  - $0° \rightarrow$ 1.5 ms pulse (7.5% duty cycle)
  - $+90° \rightarrow$ 2.0 ms pulse (10% duty cycle)

Initial Motion Sequence (Unit Test)

- $0° \rightarrow -45° \rightarrow 0° \rightarrow +45° \rightarrow 0° \rightarrow -90° \rightarrow 0° \rightarrow +90° \rightarrow 0°$
- 1 second delay between each position

Full System Behavior

- Servo follows IMU tilt angle in real-time
- Smooth continuous movement (no jumps)
- Updates as frequently as IMU data is available

Test Criteria

- PWM frequency exactly 50 Hz
- Pulse width matches expected angle (measure with oscilloscope)
- Servo moves smoothly to all positions
- No jitter or instability
- Motion sequence timing is accurate

6 Full System Integration

6.1 System Architecture Overview

The complete I2C system is organized as follows:

Processor Core (TM4C123 LaunchPad):

- WTIMER (WTIMER1–5, student choice): Provides 1 millisecond delays
- UART0: Sends debug data to PC terminal
- I2C (I2C1–3, student choice): Masters the I2C bus
- PWM (M0PWM1–7 or M1PWM0–7): Controls servo motor

I2C Slave Devices (connected to shared I2C bus):

- TCS34725 Color Sensor (address 0x29): Detects RGB colors
- MPU6050 IMU (address 0x68): Measures acceleration and motion
- 16x2 LCD with Backpack (address 0x27): Displays output

Other Devices (non-I2C):

- Servo Motor: Driven by PWM signal based on IMU data
- LED Indicators (Red, Green, Blue): Status indicators for color detection

6.2 System Data Flow

Initialization Phase

1. Initialize WTIMER (student-chosen module, not WTIMER0)
2. Initialize UART0 (debug output)
3. Initialize I2C (student-chosen module, not I2C0)
4. Detect all sensors (ID/WHO_AM_I registers)
5. Initialize PWM (student-chosen module, not M0PWM0)
6. Display startup message on LCD

Main Operating Loop

1. **Every 100 ms:** Read color and angle sensors
2. **Every 100 ms:** Calculate servo angle and update motor
3. **Every 1 second:** Format and display data on LCD and UART
4. **On Input:** Handle SW1 (start/stop detection) and SW2 (display update)

6.3 System Outputs

LCD Display

- Row 1: "Color: [RED/GREEN/BLUE]"
- Row 2: "Angle: [−90° to +90°]"
- Update frequency: 1 Hz (every 1 second)

UART Terminal

- Format: "Color: R=12345, G=02345, B=01234 | Angle: X=45°, Y=−10°, Z=2°"
- Update frequency: 1 Hz (every 1 second)
- Baud rate: Student-chosen (e.g., 115200, 9600, or 38400; not 57600)

LED Indicators

- Red LED lights when red color detected
- Green LED lights when green color detected
- Blue LED lights when blue color detected

Servo Motor

- Follows IMU tilt angle in real-time
- Responds continuously to motion
- Smooth, jitter-free movement

7 Deliverables

7.1 Required Submissions

1. **Source Code (ZIP file)**
   - All `.c` and `.h` files
   - Organized folder structure
   - Comments explaining all design decisions
   - No compiler warnings
2. **Project Report (PDF)**
   - Title page with names and date
   - System overview and architecture
   - Module descriptions and design choices
   - Hardware connection diagram
   - Software design (flowcharts, pseudocode)

- Testing procedure and results
- Conclusion and lessons learned

3. **Video Demonstration**
   - Show all modules working independently
   - Show full system integration
   - Show LCD output, UART output, servo movement
   - Explain what the system is doing during demo
4. **Test Documentation**
   - Test results for each module
   - Screenshots or oscilloscope traces if applicable
   - Evidence of successful sensor detection
   - Robustness test results (edge cases)

8 Grading Rubric

Category	Points	Criteria
Module Implementation	28	All 7 modules fully functional, no incomplete code
System Integration	21	Modules work together, proper timing, no crashes
Code Quality	11	Comments, organization, follows best practices
Documentation	7	Report explains design, test results complete
Demonstration	3	Clear video showing all features working
Participation	30	Engagement, tutoring attendance, peer collaboration
TOTAL	**100**	—

9 Notes for Students

You cannot pass with only a report—your system must work. Regardless of documentation quality, the embedded system must demonstrate all modules functioning correctly. A polished report alone cannot compensate for non-functional hardware or software.

Strong participation can save borderline projects. Students who actively engage with tutoring sessions, attend check-ins, and participate in peer collaboration have better success. Participation points can significantly boost a marginal project grade.

Weak integration will heavily impact your score even if modules work individually. Your independent modules must interact seamlessly as a single system. Systems that pass all unit tests but fail in full integration will lose substantial points during final submission, as system-level bugs are more serious than isolated module issues.

This document describes the CECS 447 Final Project I2C Bus Communication System. For questions or clarifications, contact Jackie Huynh or Dr. Min He.